Report

CS5013: Assignment 3

Parallel BFS implementation

Task 1

Design

The program works by giving the kernel current frontiers and it sees which of them are valid marks them and stores their level and then requests CPU for their neighbors. So the program runs in a wave like pattern. We start with the first node give it's neighbors to the GPU with as many thread blocks as original parents (Here 1 thread block and neighbor number of threads in each thread block). It finds the unvisited ones marks them visited stores their level and appends them to list it will forward to the CPU in the next iteration. The CPU will take these nodes find their neighbors make them into list and send them to the GPU. This way the kernel operates.

Task 2

Design

The program works by giving the kernel current frontiers and it sees which of them are valid marks them and stores their level and then requests CPU for their neighbors. So the program runs in a wave like pattern. Here we launch only limited number of threads and use 4 streams each stream runs parallel with the other and at the end all come together and synchronize. We then divide the next wave equally between all so to do this synchronization at the end of every level is needed.

Task 3

Design

It is similar to the previous it's just that this time we wrap everything in a cuda graph. We instantiate the graph every iteration and don't reus ethe graph for previyous due to new kernel arguments being there.

Speedup

Between baseline and stream a significant speedup was seen from 1500000 we went down to 1280000 microseconds. More speedup could have been there but the task 1 version is highly parallel as compared to the other two as it launches as many threads as items whereas the other two are looping. Beteen the streams and graph no speedup was seen and performamnce actually degraded a bit. This was

due to the fact that the garph was created for every iteration and nit reused this didn't guve any optimization as such but instead increased the overhead so we see a bit degraded performance from 1280000 to 1300000 microseconds